

Low-rank Connectivity and the Dynamics It Produces

A short note on the framework used by Dubreuil et al. (2022)

1 The starting point: a recurrent neural network

The paper studies firing-rate recurrent neural networks. A common continuous-time form is

$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + \sum_{s=1}^{N_{\text{in}}} I_i^{(s)} u_s(t), \quad (1)$$

where:

- $x_i(t)$ is the activation of neuron i ;
- $\phi(x_i)$ is its firing rate after a static nonlinearity;
- J_{ij} is the recurrent weight from neuron j to neuron i ;
- $u_s(t)$ are external inputs, such as stimulus or context signals;
- $I_i^{(s)}$ is the input weight from input channel s to neuron i .

In vector form,

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + J\phi(\mathbf{x}) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (2)$$

A completely general $N \times N$ recurrent matrix J can have rank as large as N . Such a matrix can create many independent recurrent feedback directions. The low-rank assumption sharply restricts this.

2 What “low rank” means

A rank- R recurrent connectivity matrix can be written as a sum of R outer products:

$$J = \frac{1}{N} \sum_{r=1}^R \mathbf{m}^{(r)} \left(\mathbf{n}^{(r)} \right)^\top, \quad (3)$$

or componentwise,

$$J_{ij} = \frac{1}{N} \sum_{r=1}^R m_i^{(r)} n_j^{(r)}. \quad (4)$$

Each term has two population-length vectors:

- $\mathbf{n}^{(r)}$ is a *read-in* or *selection* vector. It defines which population activity pattern is measured.
- $\mathbf{m}^{(r)}$ is a *write-out* vector. It defines which pattern of recurrent input is sent back into the neurons.

For rank one,

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top, \quad (5)$$

so recurrent processing has a simple form:

$$J\phi(\mathbf{x}) = \mathbf{m} \left(\frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}) \right). \quad (6)$$

The network first projects the current firing rates onto $\mathbf{n}$, producing one scalar, and then feeds that scalar back along $\mathbf{m}$.

3 The latent variable

The scalar produced by the projection is the key low-dimensional variable:

$$\kappa(t) = \frac{1}{N} \sum_{j=1}^N n_j \phi(x_j(t)). \quad (7)$$

For rank one, the recurrent input to neuron i is simply

$$m_i \kappa(t). \quad (8)$$

Thus the full N -dimensional network is not using an arbitrary recurrent input pattern. Its recurrent input always lies along the single population direction $\mathbf{m}$. For rank R , there are R such latent variables:

$$\kappa_r(t) = \frac{1}{N} \sum_{j=1}^N n_j^{(r)} \phi(x_j(t)), \quad r = 1, \dots, R. \quad (9)$$

The recurrent input is then

$$J\phi(\mathbf{x}) = \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t). \quad (10)$$

This is the main mathematical reason low-rank networks are interpretable: the large neural population can be described through a small set of recurrent latent variables $\kappa_1, \dots, \kappa_R$.

4 Why low-rank connectivity creates low-dimensional dynamics

Insert the low-rank form into the RNN:

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (11)$$

This equation says that changes in the neural state are built only from:

- the recurrent directions $\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(R)}$;
- the input directions $\mathbf{I}^{(1)}, \dots, \mathbf{I}^{(N_{\text{in}})}$;
- the leak term $-\mathbf{x}$.

If the initial state is in the subspace spanned by the recurrent and input directions, it stays there. Even when the full network has thousands of neurons, the meaningful trajectories can live in a much smaller subspace. This is what the paper means by reducing the network to latent dynamics.

5 A rank-one example

With one input $u(t)$ and rank-one recurrent connectivity,

$$\tau \dot{x}_i = -x_i + m_i \kappa(t) + I_i u(t), \quad (12)$$

$$\kappa(t) = \frac{1}{N} \sum_j n_j \phi(x_j(t)). \quad (13)$$

If ϕ were linear, the latent dynamics would be approximately

$$\tau \dot{\kappa} = -\kappa + \sigma_{nm} \kappa + \sigma_{nI} u(t), \quad (14)$$

where

$$\sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{nI} = \frac{1}{N} \sum_i n_i I_i. \quad (15)$$

These overlaps control the computation:

- σ_{nI} says how strongly the input drives the latent variable;
- σ_{nm} says how strongly the latent variable feeds back onto itself.

If feedback is weak, κ tracks the input briefly. If feedback is tuned close to the leak, κ can integrate information over time. If the nonlinearity is strong and feedback is positive, the same low-dimensional system can support attractor-like decision states.

6 Where population structure enters

The paper then asks: what statistics of the neuron-specific parameters are needed for a task? For each neuron, collect its coordinates in connectivity space:

$$\left(m_i^{(1)}, \dots, m_i^{(R)}, n_i^{(1)}, \dots, n_i^{(R)}, I_i^{(1)}, \dots, I_i^{(N_{\text{in}})}, w_i \right). \quad (16)$$

Across neurons, these coordinates form a distribution.

Single random population

In a fully random population structure, these neuron coordinates are modeled as draws from one joint distribution, often a multivariate Gaussian:

$$\mathbf{a}_i \sim \mathcal{N}(0, \Sigma). \quad (17)$$

The effective latent dynamics are governed by global covariances such as σ_{nm} and σ_{nI} . There are no distinct groups of neurons with different computational roles.

Multiple subpopulations

In a non-random population structure, the coordinates are better described as a mixture:

$$p(\mathbf{a}) = \sum_{p=1}^P \alpha_p \mathcal{N}(\mu_p, \Sigma_p). \quad (18)$$

Each component corresponds to a subpopulation. The subpopulations can have different covariances, so the same input may modulate one group strongly and another weakly.

In the nonlinear case, this is crucial. Dubreuil et al. write the effective couplings as population-weighted quantities of the form

$$\tilde{\sigma}_{ab} = \sum_p \alpha_p \sigma_{ab}^{(p)} \langle \phi' \rangle_p. \quad (19)$$

Here $\langle \phi' \rangle_p$ is the gain of subpopulation p . Inputs can change these gains differently across subpopulations. This lets context reshape the effective latent dynamics without necessarily driving the same latent variable directly.

7 The computational message

Low rank controls the *dimension* of the recurrent dynamics. A rank- R network has R main recurrent latent variables.

Population structure controls how inputs, feedback, and readout are arranged across neurons. A single random population can implement many tasks by using low-dimensional dynamics. But tasks requiring flexible input-output remapping can require multiple subpopulations, because different groups need to have different gain modulations and different effective roles.

In one sentence:

Low-rank connectivity says the network computes through a small number of collective variables; non-random population structure says different neurons are organized so that inputs can flexibly reshape the dynamics of those collective variables.

Glossary

Term	Meaning
J	recurrent connectivity matrix
rank R	number of independent recurrent feedback directions
$\mathbf{n}^{(r)}$	vector that reads population activity into latent variable r
$\mathbf{m}^{(r)}$	vector that writes latent variable r back into the population
κ_r	low-dimensional recurrent latent variable
$\mathbf{I}^{(s)}$	input vector for input channel s
$\mathbf{w}$	readout vector for producing the network output
population structure	distribution of neuron coordinates in connectivity space